

```

DROP DATABASE IF EXISTS order_processing;
CREATE DATABASE order_processing;
USE order_processing;

-- Creating Customers table
CREATE TABLE IF NOT EXISTS Customers (
    cust_id INT PRIMARY KEY,
    cname VARCHAR(35) NOT NULL,
    city VARCHAR(35) NOT NULL
);

-- Creating Orders table
CREATE TABLE IF NOT EXISTS Orders (
    order_id INT PRIMARY KEY,
    odate DATE NOT NULL,
    cust_id INT,
    order_amt INT NOT NULL,
    FOREIGN KEY (cust_id) REFERENCES Customers(cust_id) ON DELETE CASCADE
);

-- Creating Items table
CREATE TABLE IF NOT EXISTS Items (
    item_id INT PRIMARY KEY,
    unitprice INT NOT NULL
);

-- Creating OrderItems table
CREATE TABLE IF NOT EXISTS OrderItems (
    order_id INT NOT NULL,
    item_id INT NOT NULL,
    qty INT NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (item_id) REFERENCES Items(item_id) ON DELETE CASCADE
);

-- Creating Warehouses table
CREATE TABLE IF NOT EXISTS Warehouses (
    warehouse_id INT PRIMARY KEY,
    city VARCHAR(35) NOT NULL
);

-- Creating Shipments table
CREATE TABLE IF NOT EXISTS Shipments (
    order_id INT NOT NULL,
    warehouse_id INT NOT NULL,
    ship_date DATE NOT NULL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (warehouse_id) REFERENCES Warehouses(warehouse_id) ON DELETE
CASCADE
);

-- Inserting data into Customers table
INSERT INTO Customers VALUES
(1, "Customer_1", "Mysuru"),
(2, "Customer_2", "Bengaluru"),
(3, "Kumar", "Mumbai"),
(4, "Customer_4", "Delhi"),
(5, "Customer_5", "Bengaluru");

-- Inserting data into Orders table
INSERT INTO Orders VALUES
(1, "2020-01-14", 1, 2000),
(2, "2021-04-13", 2, 500),
(3, "2019-10-02", 3, 2500),

```

```

(4, "2019-05-12", 5, 1000),
(5, "2020-12-23", 4, 1200);

-- Inserting data into Items table
INSERT INTO Items VALUES
(1, 400),
(2, 200),
(3, 1000),
(4, 100),
(5, 500);

-- Inserting data into Warehouses table
INSERT INTO Warehouses VALUES
(1, "Mysuru"),
(2, "Bengaluru"),
(3, "Mumbai"),
(4, "Delhi"),
(5, "Chennai");

-- Inserting data into OrderItems table
INSERT INTO OrderItems VALUES
(1, 1, 5),
(2, 5, 1),
(3, 5, 5),
(4, 3, 1),
(5, 4, 12);

-- Inserting data into Shipments table
INSERT INTO Shipments VALUES
(1, 2, "2020-01-16"),
(2, 1, "2021-04-14"),
(3, 4, "2019-10-07"),
(4, 3, "2019-05-16"),
(5, 5, "2020-12-23");

-- Queries
SELECT * FROM Customers;
SELECT * FROM Orders;
SELECT * FROM OrderItems;
SELECT * FROM Items;
SELECT * FROM Shipments;
SELECT * FROM Warehouses;

-- Retrieve order_id and ship_date for orders shipped from Warehouse 1
SELECT order_id, ship_date FROM Shipments WHERE warehouse_id = 1;

-- Retrieve warehouses shipping orders to Kumar

SELECT o.order_id, s.warehouse_id
FROM Orders o
JOIN Customers c ON o.cust_id = c.cust_id
JOIN Shipments s ON o.order_id = s.order_id
WHERE c.cname LIKE "%Kumar%";

-- Generate customer order summary
SELECT cname, COUNT(o.order_id) AS no_of_orders, AVG(o.order_amt) AS
avg_order_amt
FROM Customers c JOIN Orders o ON c.cust_id = o.cust_id
GROUP BY cname;

--Delete all orders for customer named "Kumar"
DELETE FROM Orders
WHERE cust_id = (SELECT cust_id FROM Customers WHERE cname LIKE "%Kumar%");

```

```

-- Find item with the highest unit price
SELECT MAX(unitprice) FROM Items;

-- Create view for shipments from warehouse 2
CREATE VIEW ShipmentDatesFromWarehouse2 AS
SELECT order_id, ship_date FROM Shipments WHERE warehouse_id = 2;
SELECT * FROM ShipmentDatesFromWarehouse2;

-- A view that shows the warehouse name from where the kumar's order is been
shipped.

CREATE VIEW KumarWarehouse AS
SELECT DISTINCT w.city AS warehouse_name
FROM Warehouses w
JOIN Shipments s ON w.warehouse_id = s.warehouse_id
JOIN Orders o ON s.order_id = o.order_id
JOIN Customers c ON o.cust_id = c.cust_id
WHERE c.cname = 'Kumar';

-- Trigger to prevent warehouse deletion if shipments exist
DELIMITER $$d
CREATE TRIGGER PreventWarehouseDelete
BEFORE DELETE ON Warehouses
FOR EACH ROW
BEGIN
    IF OLD.warehouse_id IN (SELECT warehouse_id FROM Shipments) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'An item has to be shipped
from this warehouse!';
    END IF;
END;
$$
DELIMITER ;

-- Attempt to delete warehouse 2 (should fail)
DELETE FROM Warehouses WHERE warehouse_id = 2;

-- Trigger to update order amount based on quantity and unit price
DELIMITER $$
CREATE TRIGGER UpdateOrderAmt
AFTER INSERT ON OrderItems
FOR EACH ROW
BEGIN
    UPDATE Orders
    SET order_amt = (NEW.qty * (SELECT unitprice FROM Items WHERE item_id =
NEW.item_id))
    WHERE order_id = NEW.order_id;
END;
$$
DELIMITER ;

-- Insert a new order and update automatically
INSERT INTO Orders VALUES (6, "2020-12-23", 4, 1200);
INSERT INTO OrderItems VALUES (6, 1, 5);

-- Verify the updated orders table
SELECT * FROM Orders;

```